

Scheduling

Introduzione

Abbiamo visto come viene implementato il context switch ma dobbiamo ancora discutere delle policy che si trovano al di sopra di questo meccanismo. Per farlo facciamo delle assunzioni che si discostano dalla realtà: (Queste assunzioni verranno modificate per capire meglio le politiche)

Ogni job (processo) ci mette lo stesso tempo a essere eseguito,

Ogni job arriva nello stesso momento

Una volta iniziato il job dev'essere completato

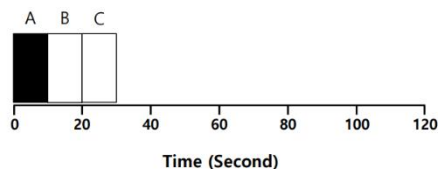
Tutti i job utilizzano solo la cpu e non I/O

Il tempo di esecuzione di ogni job è noto.

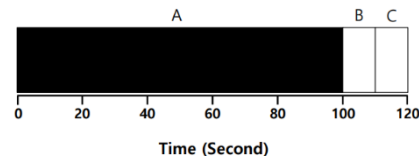
Una metrica con cui giudicheremo le varie politiche di scheduling è il Turnaround Time che sarebbe quanto tempo il processo è rimasto nel sistema ovvero $T_{\text{turnaround}} = T_{\text{Completion}} - T_{\text{arrival}}$ con le assunzioni che abbiamo fatto adesso che ogni processo arriva a tempo 0 il Turnaround è uguale al tempo di completamento del processo. Un'altra metrica è "fairness" quanto equamente le risorse vengono divise tra i processi.

FIFO/First-In-First-Out o FCFS/First-Come-First-Served

Implementare una strategia fifo può essere conveniente solo con le nostre assunzioni, infatti se i processi non arrivassero tutti a tempo zero e avessero un tempo di esecuzione differente tra loro il turnaround medio si alzerebbe troppo e si crea quello che viene detto effetto convoglio, se arriva prima il processo più lungo, quelli dopo ne devono aspettare il completamento



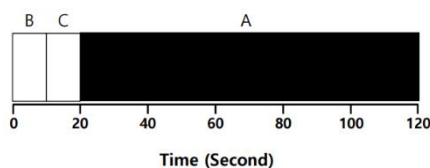
$$\text{Average turnaround time} = \frac{10 + 20 + 30}{3} = 20 \text{ sec}$$



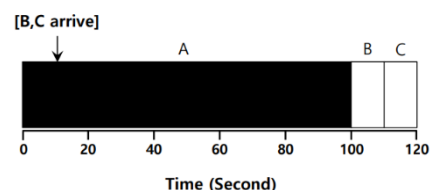
$$\text{Average turnaround time} = \frac{100 + 110 + 120}{3} = 110 \text{ sec}$$

SJF/Shortest Job First

Come fa intuire il titolo viene eseguito prima il processo più corto poi il secondo più corto e così via. La criticità di questo è simile alla FIFO, in questo caso se è presente un solo processo che è anche il più lungo e ne viene iniziata l'esecuzione e poco dopo arrivano gli altri processi, nuovamente si ritrovano bloccati dal processo più lungo.



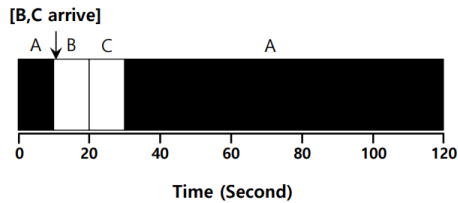
$$\text{Average turnaround time} = \frac{10 + 20 + 120}{3} = 50 \text{ sec}$$



$$\text{Average turnaround time} = \frac{100 + (110 - 10) + (120 - 10)}{3} = 103.33 \text{ sec}$$

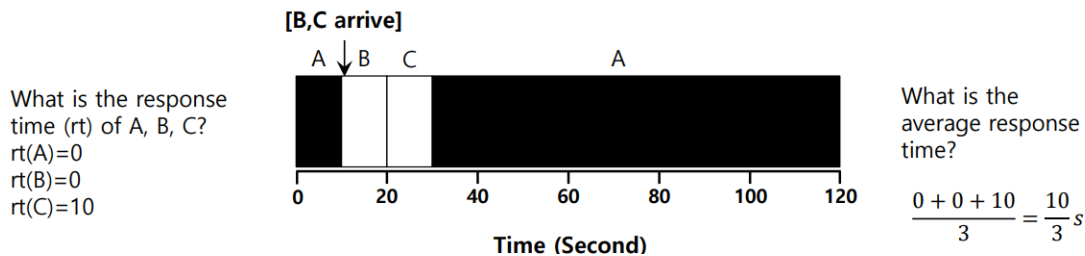
STCF/Shortest-Time-to-Completion-First

Questa politica è simile al SJF solo che non si basa sul tempo di esecuzione del processo ma sul tempo di esecuzione rimasto del processo.



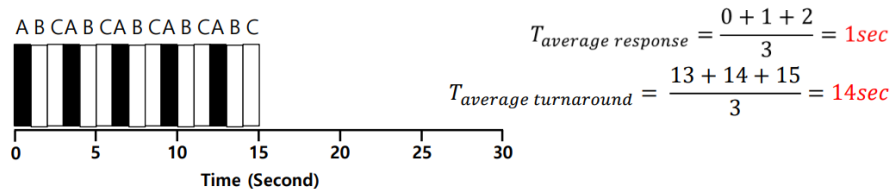
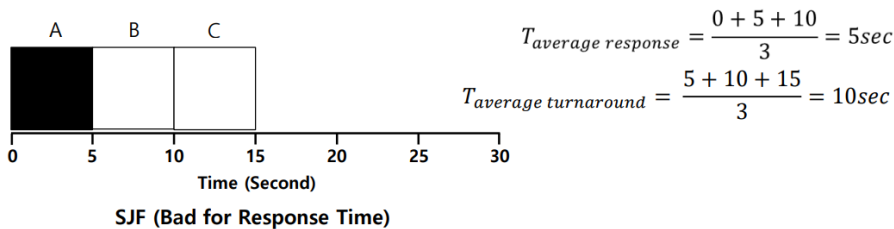
$$\text{Average turnaround time} = \frac{(120 - 0) + (20 - 10) + (30 - 10)}{3} = 50 \text{ sec}$$

Aggiungiamo un'altra metrica di giudizio che è il tempo di risposta, che consiste nel tempo che quel job è stato in coda prima di eseguire la prima istruzione ed è uguale a $T_{\text{response}} = T_{\text{firstrun}} - T_{\text{arrival}}$



RR/Round Robin

Questa politica è basata su fette temporali, ovvero un job va in esecuzione per un certo quanto di tempo, successivamente viene mandato in esecuzione un altro job fino a che non c'è più nulla in coda. Questa politica è fair ma così facendo ottiene un turnaround pessimo.



RR with a time-slice of 1sec (Good for Response Time)

Questa politica sia con un quanto di tempo lungo che corto ha sempre delle criticità. Più il quanto di tempo è piccolo e più il costo del context switch dominerà le performance. Più il quanto di tempo è grande e peggiore sarà il tempo di risposta. Generalmente ogni politica fair avrà un cattivo turnaround time. Se volessimo incorporare anche l'I/O dovremmo modificare qualche assunzione, del tipo i processi possono usare I/O, Quando un job inizia una richiesta di I/O: Il job rimane bloccato e aspetta

che l'I/O finisca, il gestore dovrebbe gestire un altro job sulla cpu. Quando l'I/O finisce viene sollevato un Interrupt e il sistema operativo modifica lo stato del processo da bloccato a pronto

